

Eventos, entrega y semánticas de procesamiento

Streaming de datos y sus aplicaciones

Maestría en Inteligencia Artificial

Facultad Politécnica - Universidad Nacional de Asunción

Rodrigo Parra, M.Sc.

Clase 3

Contents

1. Objetivos de la clase

- Conexión con la clase anterior
- La pregunta que quedó abierta
- ¿Qué queremos lograr hoy?
- Por qué importa para aplicaciones de AI
- Cinco términos que usaremos durante la clase

2. El evento como contrato

- Un evento afirma que algo ocurrió

- Cuatro campos, cuatro responsabilidades
- Identidad del evento
- Igual payload no significa mismo evento
- Evento, comando y estado no son lo mismo

3. Fallos, reintentos e incertidumbre

- Un timeout no revela qué ocurrió
- La ventana de ambigüedad
- Entrega y procesamiento son capas distintas
- Fallos que debemos imaginar

4. Tres semánticas de procesamiento

- Una lista de tareas explica el problema

- Las tres promesas
- At-most-once: confirmar antes de procesar
- Cuándo aceptar at-most-once
- At-least-once: procesar antes de confirmar
- At-least-once mueve el problema
- Exactly-once: una propiedad observable
- Comparación rápida
- Actividad: elegir el error tolerable

5. Diseñar para reintentos

- Idempotencia: repetir sin cambiar el resultado
- La clave de idempotencia

- Upsert: repetir converge al mismo estado
- El SQL implementa esa convergencia
- Un upsert no corrige eventos fuera de orden
- Deduplicación: recordar qué ya vimos
- Inbox responde: ¿ya procesé este evento?
- Inbox funciona dentro de una transacción local
- Outbox evita una actualización sin evento
- Outbox guarda el cambio y la publicación pendiente
- El relay puede publicar más de una vez
- Inbox y outbox protegen lados opuestos
- La deduplicación necesita una frontera temporal

- Idempotencia y deduplicación se complementan

6. Progreso, estado y recuperación

- El commit registra progreso, no el efecto
- Checkpoint: una fotografía consistente
- Recuperar significa volver a un punto coherente
- Procesamiento determinista facilita el replay
- Errores de datos no son fallos transitorios

7. Exactly-once y sus límites

- Kafka puede coordinar Kafka $\rightarrow$ Kafka
- Productor idempotente no equivale a pipeline exactly-once
- El límite aparece al salir del sistema

- Opciones para efectos externos
- Exactly-once debe escribirse con alcance
- Trade-offs de garantías más fuertes

8. Práctica guiada

- Caso: scoring de transacciones
- Arquitectura de la práctica guiada
- Mapa de fallos del pipeline
- Diseño propuesto
- Pseudocódigo tolerante a reintentos
- Inyección de fallos
- Checklist de diseño

9. Síntesis y cierre

- Volvamos al fallo inicial
- Cinco ideas para llevar
- Lecturas para consolidar
- Puente hacia la clase 4
- Fuentes y atribuciones

Objetivos de la clase

Clase 2

- Kafka conserva eventos en un log
- los consumidores avanzan con offsets
- la retención permite replay
- el orden existe por partición

Clase 3

- qué significa “procesar” un evento
- cuándo confirmar el offset
- pérdidas, duplicados y reintentos
- idempotencia, deduplicación y recuperación

Idea clave

El log permite repetir la lectura; la aplicación debe decidir qué significa repetir el procesamiento.

```
for record in consumer.subscribe("clickstream.raw"):
    event = parse(record.value)
    update_metric(event)
    commit_offset(record)
```

Escenario de fallo

`update_metric(event)` termina correctamente, pero el proceso falla antes de `commit_offset(record)`.

¿Perdemos el evento, lo repetimos o lo procesamos exactamente una vez?

Al final de esta clase deberías poder

- Modelar un evento con identidad, clave, tiempo y payload.
- Comparar **at-most-once**, **at-least-once** y **exactly-once**.
- Explicar cómo la posición del commit cambia la garantía.
- Reconocer duplicados originados por reintentos y recuperación.
- Proponer estrategias de idempotencia y deduplicación.
- Delimitar qué garantiza una plataforma y qué depende del destino.

Pregunta guía: ¿qué resultado observable queda después de un fallo y un reinicio?

Un duplicado puede

- contar dos veces una interacción
- emitir dos alertas
- cobrar dos veces una acción
- actualizar dos veces una feature

Una pérdida puede

- ocultar una anomalía
- omitir una señal de fraude
- sesgar métricas online
- romper un feedback loop

Idea clave

La semántica de procesamiento es parte del significado del resultado, no un detalle de infraestructura.

Término	En palabras simples	Ejemplo
Offset	posición dentro del log	42: el próximo evento está en esa posición
Commit	guardar el progreso	“ya terminé hasta el evento 42”
Reintento	volver a intentar tras un fallo	reenviar porque no llegó confirmación
Idempotencia	repetir sin cambiar el resultado	marcar una orden como PAID otra vez
Efecto externo	acción fuera del procesador	guardar en una DB, cobrar o enviar un email

Idea clave

Primero razonaremos con estas ideas; después veremos los mecanismos técnicos.

El evento como contrato

Definición operativa

Un evento es un registro inmutable de un hecho relevante para el dominio, identificado y ubicado en el tiempo.

event_id: `evt-9f2a`

key: `account-42`

timestamp: `2026-07-17T14:05:31Z`

payload: `{type: payment, amount: 250000}`

Campo	Responsabilidad	Pregunta de diseño
event_id	identidad lógica del hecho	¿cómo detectamos una repetición?
key	afinidad, orden y estado por entidad	¿qué eventos deben viajar juntos?
timestamp	momento relevante del dominio	¿cuándo ocurrió realmente?
payload	datos necesarios para interpretar el hecho	¿el contrato puede evolucionar?

Buena identidad

- estable entre reintentos
- única dentro del dominio
- generada cerca del origen
- propagada a eventos derivados

Identidad frágil

- cambia en cada reintento
- depende del consumidor
- se calcula con campos mutables
- se pierde al transformar

Idea clave

Sin una identidad estable, distinguir un duplicado de dos hechos iguales puede ser imposible.

```
event_id: `evt-101`  
key: `sensor-7`  
timestamp: `10:00:01`  
payload: `{temperature: 28}`
```

```
event_id: `evt-102`  
key: `sensor-7`  
timestamp: `10:00:05`  
payload: `{temperature: 28}`
```

Interpretación

Son dos mediciones distintas con el mismo valor. Deduplicar por payload eliminaría información válida.

Comando

“Cobrar G.
250.000”

Solicita que algo
ocurra.

Evento

“Pago aprobado”

Registra un hecho
ocurrido.

Estado

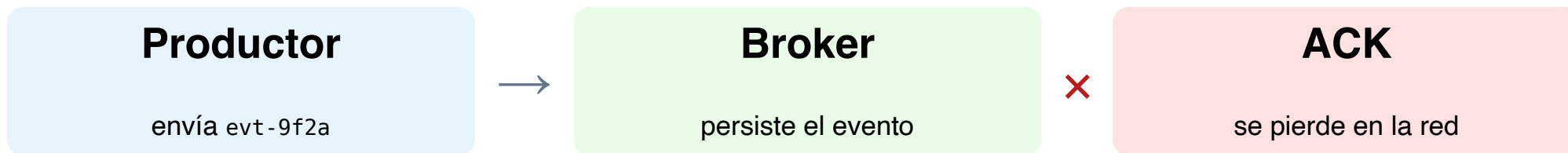
“Saldo actual”

Resume hechos
hasta un punto.

Idea clave

Las garantías deben formularse sobre el efecto observable, no solamente sobre el mensaje transportado.

Fallos, reintentos e incertidumbre



Desde el productor

No llegó confirmación. Reintentar evita una posible pérdida, pero puede repetir un envío que sí había sido confirmado por el broker.

Antes del efecto

Si el proceso falla antes de guardar el resultado, el evento debe volver a procesarse.

Después del efecto

Si falla después de guardar el resultado pero antes de registrar progreso, el evento reaparece.

Entre **efecto confirmado** y **progreso confirmado** vive el riesgo de duplicación.

Capa	Pregunta	Mecanismo típico
Publicación	¿el evento quedó en el log?	confirmaciones (acks), réplica y reintento
Consumo	¿desde qué posición reinicia?	offsets y commits
Procesamiento	¿el estado interno es consistente?	checkpoint y restauración
Salida	¿el efecto externo se repite?	transacción, idempotencia o deduplicación

Idea clave

“Exactly once” siempre necesita un alcance explícito: ¿exactamente una vez dónde?

Infraestructura

- pérdida temporal de red
- caída de broker
- cambio de líder
- rebalanceo de consumidores

Aplicación

- excepción al transformar
- caída después de escribir
- timeout del destino
- reinicio desde un checkpoint

Método de análisis

Para cada frontera: **¿qué ya ocurrió?, ¿qué quedó confirmado y qué se repetirá al reiniciar?**

Tres semánticas de procesamiento

Tarea: llamar a un cliente

Tenemos dos acciones: **hacer la llamada** y **marcar la tarea como completada**. El orden importa si la persona se interrumpe entre ambas.

Marcar y luego llamar

Si hay un fallo después de marcar, nadie retoma la llamada: podemos **perderla**.

Llamar y luego marcar

Si hay un fallo antes de marcar, alguien vuelve a llamar: podemos **repetirla**.

Idea clave

En streaming, “marcar la tarea” equivale a confirmar el offset.

At-most-once

Promesa: cero o una ejecución

Costo/riesgo: puede perder eventos

At-least-once

Promesa: una o más ejecuciones

Costo/riesgo: puede duplicar efectos

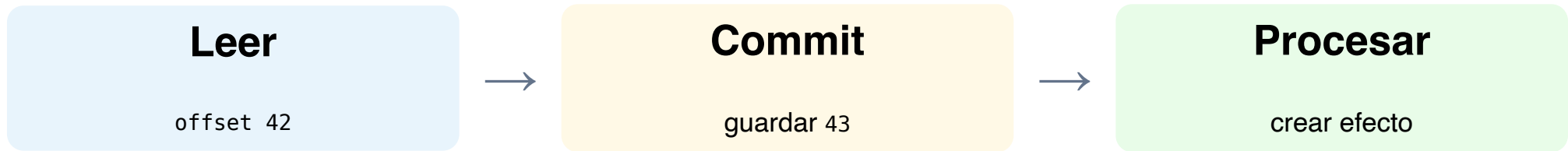
Exactly-once

Promesa: un efecto lógico observable

Costo/riesgo: requiere coordinación

Idea clave

No existe una semántica universalmente “mejor”; existe una semántica compatible con el costo del error.



Si el proceso cae después del commit

Al reiniciar comienza en 43; el evento 42 no se vuelve a entregar y su efecto puede perderse.

Puede ser razonable

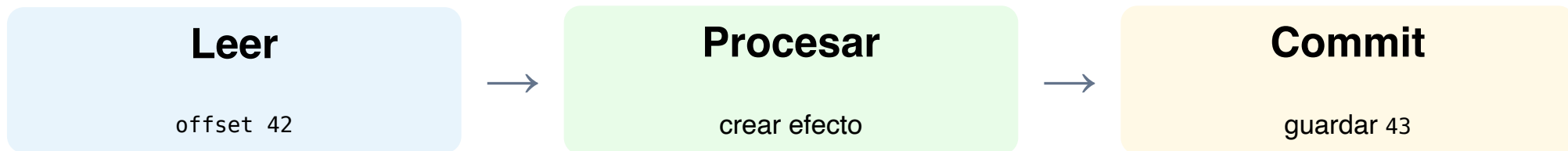
- telemetría de alta frecuencia
- métricas aproximadas
- datos reemplazados pronto
- latencia mínima prioritaria

Suele ser inaceptable

- pagos y facturación
- auditoría
- inventario
- alertas críticas

Idea clave

La pérdida debe ser una decisión explícita respaldada por el dominio.



Si el proceso cae antes del commit

Al reiniciar vuelve a leer 42. El evento no se pierde, pero el efecto puede ejecutarse otra vez.

Infraestructura

garantiza reintento



Aplicación

tolera repetición

Diseño frecuente

Elegir at-least-once y hacer idempotentes los efectos suele ser más simple que coordinar un commit único entre varios sistemas.

Definición útil

Aunque la infraestructura reintente y el código se ejecute varias veces, el estado o resultado visible equivale a haber aplicado cada evento lógico una sola vez.

No significa

que cada instrucción de CPU se ejecuta una sola vez.

Sí significa

que los reintentos no producen efectos lógicos adicionales dentro del alcance declarado.

Semántica	Pérdida	Duplicado	Complejidad
At-most-once	posible	no por redelivery	baja
At-least-once	evitada dentro del supuesto	posible	media
Exactly-once	evitada dentro del alcance	oculto/neutralizado	alta o delegada

La letra pequeña

Toda garantía depende de supuestos: durabilidad, retención, disponibilidad del log, política de errores y comportamiento del destino.

Caso	¿Pérdida o duplicado?	Estrategia inicial
temperatura cada segundo	?	?
orden de cobro	?	?
email promocional	?	?
alerta de fraude	?	?
actualización de feature online	?	?

Para cada caso: justificar el costo de perder y el costo de repetir.



10 minutos

Al volver: cómo diseñar operaciones que toleren reintentos.

Diseñar para reintentos

Idempotente

```
set_status(order_id, "PAID")
```

Aplicarlo nuevamente deja el mismo estado.

No idempotente

```
increment_balance(account,  
250000)
```

Cada repetición cambia el estado.

Idea clave

La idempotencia depende de la operación y de su clave, no solo del transporte.

```
charge(  
  account_id="account-42",  
  amount=250000,  
  idempotency_key="evt-9f2a",  
)
```

Contrato

El destino recuerda el resultado asociado a evt-9f2a. Si recibe la misma clave, devuelve el resultado previo sin ejecutar otro cobro.

¿Qué significa upsert?

Si la fila no existe, la crea (**insert**).

Si ya existe, la actualiza (**update**).

La clave `id='o-7'` identifica una sola orden.

Resultado observable

1ª ejecución: o-7 → PAID

2ª ejecución: o-7 → PAID

N ejecuciones: o-7 → PAID

Idea clave

El evento expresa un estado deseado, no una acción acumulativa. Repetirlo no crea otra orden ni suma otro pago.

Operación

```
INSERT INTO orders(id,  
status)  
VALUES ('o-7', 'PAID')  
ON CONFLICT (id)  
DO UPDATE SET status =  
'PAID';
```

Por qué ayuda

- clave natural estable
- el conflicto reutiliza la fila
- el reintento converge
- el estado final es explícito

Contraste: `balance = balance + 250000` no es idempotente; cada repetición vuelve a sumar.

Historia real

10:00 — orden PAID

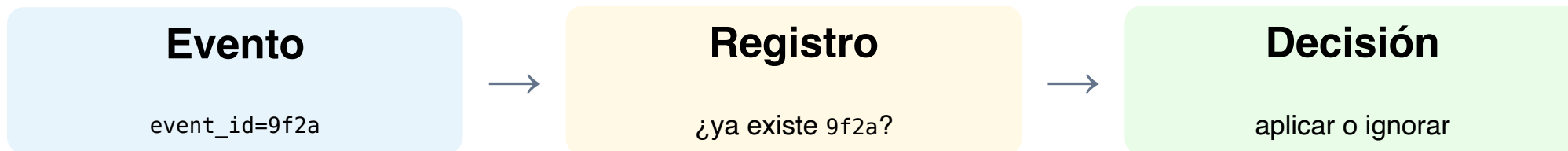
10:05 — orden REFUNDED

Replay problemático

A las 10:10 reaparece el evento antiguo PAID y sobrescribe el estado más nuevo.

Protección adicional

Guardar `event_time` o una versión y actualizar únicamente si el evento recibido es posterior al estado almacenado.



Costo oculto

La deduplicación necesita estado: almacenamiento, consultas y una política de retención.

Problema

El consumidor actualiza la orden, falla antes de confirmar el offset y recibe nuevamente evt-9f2a.

Sin inbox

No existe memoria de la primera ejecución. El cambio de negocio podría repetirse.

Con inbox

Una tabla registra evt-9f2a. Si reaparece, sabemos que ya fue aplicado.

Idea clave

Inbox protege la **entrada**: deduplica eventos recibidos.

```
BEGIN;  
  
WITH received AS (  
  INSERT INTO inbox(event_id)  
  VALUES ('evt-9f2a')  
  ON CONFLICT DO NOTHING  
  RETURNING event_id  
)  
UPDATE orders SET status = 'PAID'  
WHERE id = 'o-7'  
  AND EXISTS (SELECT 1 FROM received);  
  
COMMIT;
```

Condición crítica

Registrar `event_id` y cambiar la orden deben confirmarse juntos: o suceden ambos, o no sucede ninguno.

`received` contiene una fila solo la primera vez. Si el evento se repite, el `UPDATE` no encuentra esa evidencia y no se ejecuta.

El problema del dual write

1. La base confirma que la orden está PAID.
2. La aplicación intenta publicar `OrderPaid` en Kafka.
3. La red falla: la base cambió, pero otros sistemas nunca reciben el evento.

Idea clave

Dos escrituras independientes no forman una sola operación atómica.

Una transacción en la base

```
BEGIN;  
UPDATE orders  
  SET status = 'PAID'  
  WHERE id = 'o-7';  
INSERT INTO outbox(...)   
  VALUES ('OrderPaid', ...);  
COMMIT;
```

Después del commit

Un **relay** —proceso que reenvía pendientes— lee la fila de outbox y la publica en Kafka.

Si Kafka no está disponible, la fila permanece para reintentar.

Idea clave

Outbox protege la **salida**: evita perder el evento que debemos publicar.



Fallo posible

Kafka recibe el evento, pero el relay falla antes de marcar la fila como enviada. Al reiniciar, vuelve a publicarla.

Por eso outbox suele ofrecer **at-least-once**: el consumidor siguiente necesita idempotencia o inbox.

Patrón	Pregunta	Qué protege
Inbox	¿ya procesé este event_id?	la entrada frente a entregas repetidas
Outbox	¿cómo publico después del cambio?	la salida frente a fallos entre DB y Kafka

Juntos en una transacción

Registrar inbox + cambiar la orden + crear outbox. El commit local confirma los tres o ninguno.

Guardar para siempre

- costo creciente
- índices más pesados
- operación compleja

Expirar

- define ventana de dedup
- reaparece riesgo histórico
- debe superar el máximo reintento/replay esperado

Pregunta de diseño

¿Cuánto tiempo después puede reaparecer legítimamente el mismo event_id?

Estrategia	Ventaja	Límite
Idempotencia	repetir converge al mismo resultado	no todas las operaciones pueden reformularse
Deduplicación	bloquea eventos ya observados	requiere estado y retención
Transacción	coordina cambios atómicamente	alcance limitado y mayor acoplamiento

Progreso, estado y recuperación

Offset comprometido

Indica desde qué posición debe reanudar el consumidor.

Resultado externo

Vive en una base, API, tópico, archivo, alerta u otro sistema.

Idea clave

Confirmar un offset no deshace ni confirma automáticamente lo que ocurrió fuera de Kafka.

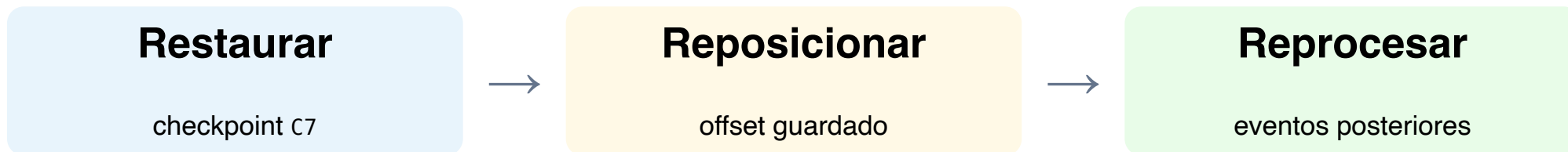
Un checkpoint coordina

- posición de entrada
- estado interno del operador
- temporizadores o metadatos relevantes
- información necesaria para reanudar

entrada hasta 42

estado = S42

checkpoint C7



Resultado

El estado interno puede reconstruirse correctamente aunque algunas operaciones vuelvan a ejecutarse.

Determinista

mismo evento + mismo estado

→ mismo resultado

No determinista

depende de reloj, aleatoriedad,
API mutable o estado no
registrado

→ resultado distinto

Idea clave

Si el replay puede producir otra respuesta, la recuperación deja de ser una reproducción fiel.

Fallo transitorio

timeout, red, servicio saturado

Respuesta: reintento con backoff.

Evento inválido

esquema incompatible, campo imposible

Respuesta: cuarentena o cola de eventos rechazados (**dead-letter queue**, DLQ).

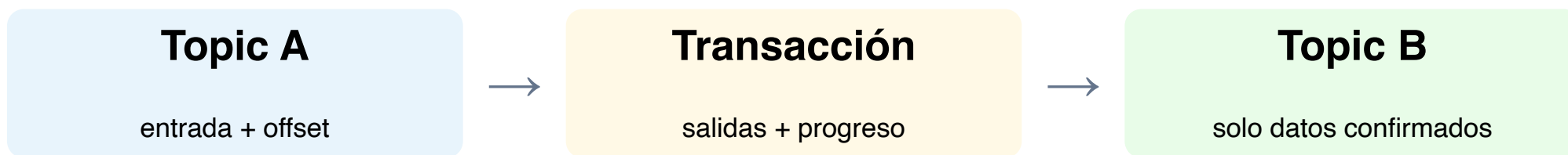
Riesgo

Reintentar infinitamente un evento “venenoso” bloquea progreso y amplifica efectos secundarios.

Exactly-once y sus límites

Transacción en Kafka

Un procesador puede escribir resultados en tópicos y confirmar offsets de entrada en la misma transacción.



Productor idempotente

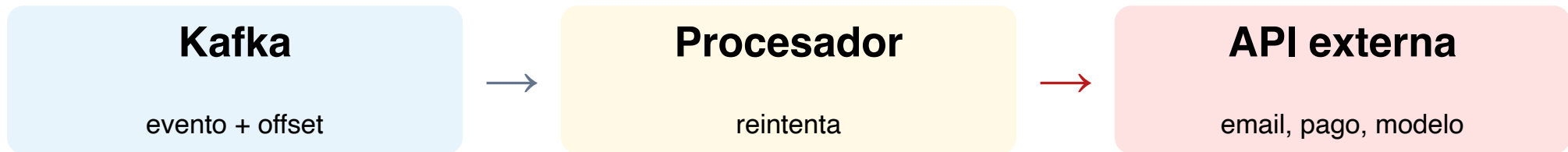
Evita que reintentos del productor escriban duplicados en una partición.

Pipeline end-to-end

Debe coordinar entrada, estado, salida y cualquier efecto externo.

Idea clave

Una garantía local no se vuelve automáticamente una garantía de negocio.



Problema

La transacción del log no puede deshacer un email enviado ni obligar a una API arbitraria a participar en el mismo commit.

Opción	Cómo funciona	Cuándo usar
Clave idempotente	el destino reconoce el mismo intento	APIs que ofrecen este contrato
Inbox / dedup	el destino registra <code>event_id</code>	base controlada por el equipo
Outbox	separa commit local de publicación	doble escritura: DB + log
Compensación	otro evento revierte o neutraliza	no existe atomicidad
Aceptar duplicado	se observa y reconcilia	costo bajo y controlado

Afirmación débil

“El sistema tiene exactly-once.”

Afirmación verificable

“Para eventos consumidos de `payments.raw`, el estado y los offsets se confirman atómicamente; la API de cobro deduplica por `event_id` durante 30 días.”

Idea clave

Una garantía útil nombra frontera, mecanismo, duración y supuesto de fallo.

Coordinación

más protocolos,
estado y
acoplamiento

Latencia

commits, barreras
y confirmaciones

Operación

recuperación,
métricas y pruebas
de fallo

Criterio

Comprar una garantía más fuerte solo cuando reduce un riesgo de negocio mayor que su costo técnico y operativo.

Práctica guiada

Escenario

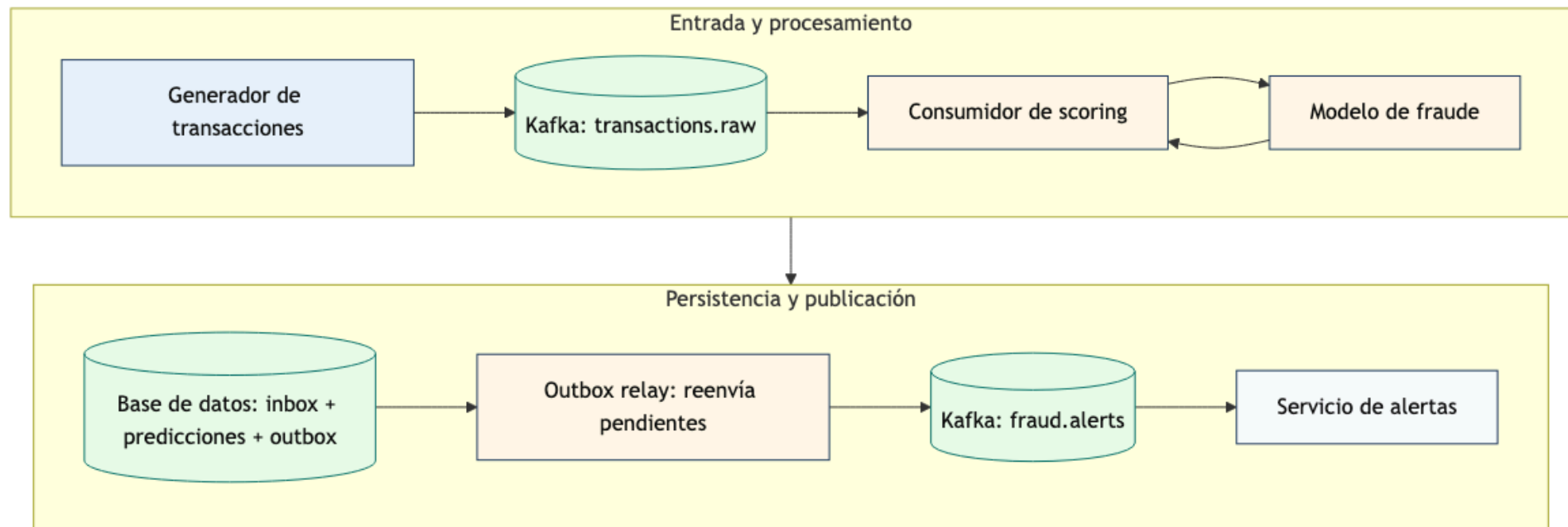
Una entidad financiera recibe transacciones continuamente. Para cada una, un modelo estima el riesgo de fraude entre 0 y 1. Si el riesgo es alto, el sistema debe generar una alerta.

Entrada: una transacción

- event_id
- account_id
- monto y comercio
- event_time

Resultados esperados

- fila de predicción
- evento fraud.scores
- alerta si supera el umbral



Transacción local: inbox, predicción y outbox se confirman juntos.

Offset: se confirma después de que la base acepta la transacción.

Frontera	Fallo	Consecuencia posible
consumo → inferencia	caída antes del score	reprocesamiento
inferencia → base	timeout ambiguo	fila ausente o repetida
base → commit	caída tras insertar	mismo evento vuelve
score → alerta	reintento	dos alertas
modelo remoto	versión cambia	replay produce otro score

Decisiones

- Consumo **at-least-once**: ante la duda, repetir.
- `event_id` estable desde el productor.
- Predicción persistida con clave única (`event_id`, `model_version`).
- Upsert idempotente: repetir no agrega otra predicción.
- Alerta publicada mediante outbox: no se pierde tras el commit.
- Consumidor de alertas con inbox o clave idempotente.
- Versión del modelo registrada en cada resultado.

Idea clave

El objetivo no es evitar toda repetición interna, sino neutralizar sus efectos observables.

```
event = parse(record.value)
score = model.predict(event.features)

with database.transaction():
    inserted = inbox.try_insert(event.event_id)
    if inserted:
        predictions.upsert(event.event_id, MODEL_VERSION, score)
        outbox.add(alert_for(event, score))

consumer.commit(record.offset + 1)
```

La transacción local coordina inbox, predicción y outbox. El commit del offset ocurre después.

Probar deliberadamente

- caída antes de escribir el resultado
- caída después del commit de la base
- timeout al publicar la alerta
- reinicio desde un offset anterior
- repetición manual del mismo `event_id`
- replay con una nueva versión del modelo

Una garantía no está completa hasta que puede demostrarse bajo fallos reproducibles.

Semántica

- ¿qué puede perderse?
- ¿qué puede repetirse?
- ¿cuál es el alcance?
- ¿cuándo se confirma progreso?

Mecanismos

- identidad estable
- idempotencia o deduplicación
- checkpoint consistente
- política de errores
- pruebas de recuperación

Síntesis y cierre

Fallo

`update_metric(event)` terminó y el consumidor cayó antes de confirmar el offset.

At-most-once

commit antes:
podría perder la
actualización.

At-least-once

commit después:
repetirá el evento.

Efecto único

actualización
idempotente o
deduplicada.

Idea clave

La respuesta correcta no es “Kafka lo resuelve”; es explicar el orden de acciones y el mecanismo del destino.

Hoy

- Todo evento necesita identidad y contrato claros.
- El commit decide desde dónde se recupera, no qué efecto externo ocurrió.
- At-most-once acepta pérdida; at-least-once acepta repetición.
- Exactly-once es una propiedad observable dentro de una frontera.
- Idempotencia, deduplicación y transacciones convierten reintentos en resultados correctos.

Bibliografía base

- Tyler Akidau, Slava Chernyak y Reuven Lax — **Streaming Systems**.
- Martin Kleppmann — **Designing Data-Intensive Applications**, capítulos sobre confiabilidad, logs y consistencia.
- Apache Kafka — [Message Delivery Semantics](#).
- Jay Kreps — [The Log](#).
- Tyler Akidau — [Streaming 101](#) y [Streaming 102](#).

Hasta ahora

Podemos repetir eventos sin perder consistencia, pero aún suponemos que sabemos cuándo incluirlos en un resultado.

Próxima pregunta

¿Cómo calculamos resultados correctos cuando los eventos llegan tarde, desordenados y sin un final natural?

Tiempo de evento · ventanas · watermarks · datos tardíos

- Objetivos, temas y resultados esperados: syllabus de la asignatura.
- Semánticas de entrega, offsets, productor idempotente y transacciones: [Apache Kafka — Design: Message Delivery Semantics](#).
- Modelo conceptual de eventos: [Apache Kafka — Introduction](#).
- Fundamentos de procesamiento, corrección y tolerancia a fallos: Akidau, Chernyak y Lax, **Streaming Systems**.
- Confiabilidad, logs, consistencia e idempotencia: Kleppmann, **Designing Data-Intensive Applications**.
- Log, replay y procesamiento de streams: Jay Kreps, [The Log](#).
- Diagrama de la práctica guiada: elaboración propia con Mermaid.

Uso didáctico

Los ejemplos fueron adaptados al curso para razonar sobre fallos, recuperación y efectos observables.

¿Preguntas?

 rmparra@pol.una.py

 **Bibliografía base: Streaming Systems + Designing Data-Intensive Applications**

 **Próxima clase: Tiempo de evento, ventanas y marcas de agua**